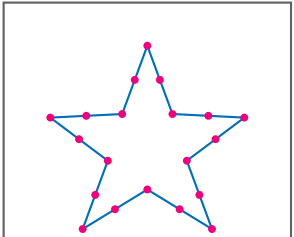
# MESHES FROM IMAGES: MESHMERIZEME

From a photograph to Lagrangian points at $\Delta s = \Delta x/2$: ContourizeMe's contours, SVG paths and Bézier curves, MeshmerizeMe's equal-distance points; checked against the files IB2d ships, reproduced with a small sampler, and run as a starfish in a channel



SVG path → points at $\Delta s = \Delta x/2$

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part II • Chapter 4f

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

# WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

- Describe the MeshmerizeMe workflow: an image, contours found by ContourizeMe, an SVG file cleaned or drawn in a vector editor, and MeshmerizeMe's `.vertex` file at the spacing $\Delta s = \Delta x/2$ the immersed boundary method wants.
- Read SVG path data (the commands, relative coordinates, the smooth curves, the arcs), evaluate Bézier curves and their arc length, and map SVG coordinates to the simulation box.
- Explain how MeshmerizeMe places the points (equal Euclidean distances, found by gradient descent) and what its errors are; check a `.vertex` file's spacing yourself.
- Turn a `.vertex` file into an IB2d example: targets, walls, a driven flow; and know what the IB2d starfish example does with its points, and what it does not.

> MeshmerizeMe itself is not installed on this tutorial's laptop. This chapter describes it from its paper, the 2021 lecture and Senter's design notes, measures the MeshmerizeMe output that ships with IB2d, and does the same job (points at equal Euclidean distances, found by a walk and bisection instead of MeshmerizeMe's gradient descent) with a 400-line script (`examples/IB2d/Mesh_From_SVG/svg_to_vertex.py`, about 320 lines of code) whose output then runs in Python IB2d (October 2026).

# WHERE THIS CHAPTER COMES FROM

- "MeshMerizeMe" (June 2021, 7 slides): the project's GitHub page (https://github.com/dmsenter89/MeshmerizeMe, "Software to create 2D geometry for IBAMR and IB2D"), installation, ContourizeMe's usage, MeshmerizeMe's usage, each pointing to the project's documentation pages (slide 4: "See the project wiki"); and the paper.
- D. M. Senter, D. R. Douglas, W. C. Strickland, S. G. Thomas, A. M. Talkington, L. A. Miller, N. A. Battista, "A semi-automated finite difference mesh creation method for use with immersed boundary software IB2d and IBAMR", Bioinspir. Biomim. 16 (2021) 016008: the workflow, ContourizeMe's image processing, MeshmerizeMe's discretisation and its errors, and three examples.
- M. Senter, "Creating a Curvilinear Mesh from Bézier Curves" (design notes for an early C++ version, UNC, about 2017, 20 pages): Bézier curves, de Casteljau, arc length, SVG path data and primitives, the map from SVG space to the experiment.
- IB2d's `matIB2d/Examples/Example_Meshmerize_Me`: the dragonfly wing, insect wing veins, leaf and starfish examples, their SVG files and MATLAB scripts, the MeshmerizeMe output for the starfish and the wing, and an unfinished `MeshingTutorial.md`.

What is new here: the starfish files measured; the design notes' arc-length rule tested; a small sampler written, tested on IB2d's SVGs, and used to build the starfish example in Python, which ran; four things about the shipped starfish example found on the way.

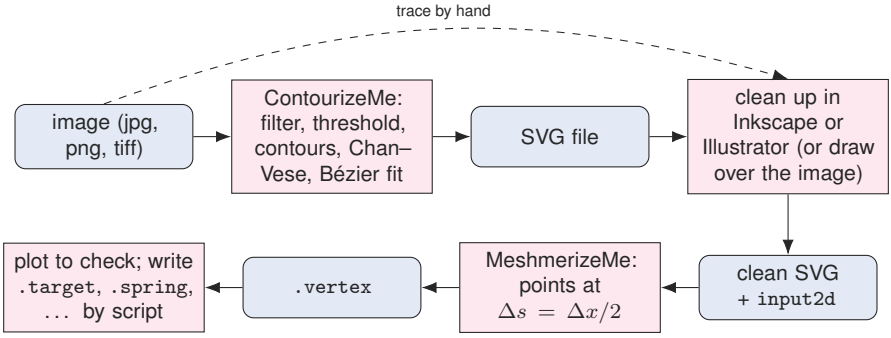
# WHY A MESHER FOR THE IMMERSED BOUNDARY METHOD

- Many IB studies are done in 2D with shapes that have formulas (plates, strings, tubes, ellipses, hemi-ellipses, circles); anything else has to be built point by point or from piecewise functions, which is “non-trivial”. The paper quotes a Pixar scientist: “I hate meshes. I cannot believe how hard this is.”
- Mesh generators (MeshLab, Gmsh, TetGen) make finite element meshes (chapter 18). The classic IB method needs a finite difference curve: points along the boundary about $\Delta s = \Delta x/2$ apart, in order, in the simulation’s coordinates.
- Biology often comes with images: a photograph, a micrograph, a scan. The geometry is in the picture; the work is getting it out.

The paper’s examples: hemolymph through dragonfly wing veins (IB2d, a sweep of $Re$ over $[0.1, 100]$, shown at 0.6, 6 and 60), lymph through a branching lymphatic capillary of a mouse embryo (IBAMR), oscillatory flow past one, three and five starfish (IB2d, $Re = 800$); and, before the paper, blood flow over the trabeculae of zebrafish embryonic hearts.

Senter et al. (2021), sections 1, 3 and 4.

# THE WORKFLOW




Two entry points: automatic edge detection with ContourizeMe, or an SVG drawn by hand over the image (when there is no image, the image is poor, or the shape is hypothetical). Either way the SVG is the interface: any vector editor can fix it. MeshmerizeMe only makes the points; which points are springs, beams or targets is written by a script afterwards (a few of these relations are classes in the MeshmerizeMe library).

After figure 1 of Senter et al. (2021) (blue: files; red: actions), redrawn.

# CONTOURIZEME: FROM PIXELS TO CURVES

A Tkinter GUI started from the command line on an image (jpg, png, tiff):

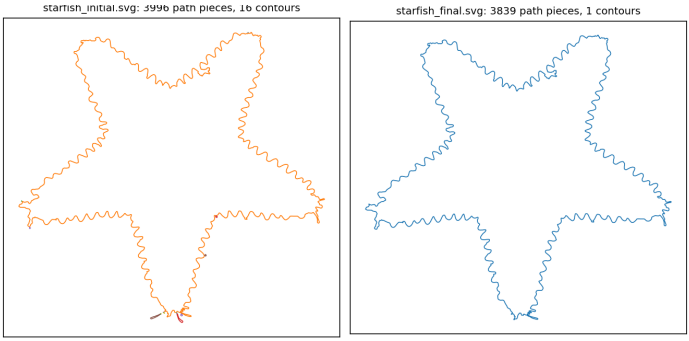
```
ContourizeMe image.jpg
# usage: ContourizeMe [-h] image
```

- choose a parameterisation of the pixels: grayscale, RGB or HSV; sliders set the threshold bounds (one for grayscale, three for RGB and HSV) and update the detected edges live (green);
- optional contrast and saturation changes (temporary) and noise reduction: average, Gaussian, median, or the recommended bilateral filter (it weights pixels by intensity difference as well as distance, to keep edges);
- the thresholded binary image goes to OpenCV's topological contour algorithm (integer pixel boundaries), refined to sub-pixel with the Chan–Vese algorithm at a chosen smoothness; Bézier curves through evenly spaced points of the refined boundary are written to an SVG file.

What makes a good image: a strong contrast between the object and the background, and little noise; otherwise the sliders and filters, then deleting stray edges by hand.
What comes out: one tiny path per cubic piece. The starfish of the IB2d example: 3996 `<path>` elements, each `M x,y C ...`, with `stroke-width` and `fill="none"`; the coordinates are the image's pixels, but the drawing's `viewBox` is the contours' bounding box enlarged by about 20 % (a 10 % margin on each side), not the image's frame: the starfish's starts at $(102.08, 20.94)$, the dragonfly wing's at $x = -46.75$.
The IB2d dragonfly example's folder keeps the intermediate pictures: `Images/cropped_wing_v3.png`, the Chan–Vese result `chanvese_contour.png`, and the SVG.

The 2021 lecture, slide 6; Senter et al. (2021), section 2.1 and figure 2.

# CLEANING THE SVG: THE STARFISH BEFORE AND AFTER




Left: ContourizeMe's output (`starfish_initial.svg`): 3996 pieces that join into 16 contours, the starfish (5950 SVG units long) and 15 small loops of noise (73 units and less). Right: the cleaned file (`starfish_final.svg`): 3839 pieces, one closed contour. (The cleaned version saved by Inkscape, `starfish_cleaned.svg`, holds the same curve.)

The paper's advice: delete curves that come from background noise; collapse groups and simplify contiguous paths (a "save" option of some editors, or SVGO / SVGOMG); give each object one contiguous path. MeshmerizeMe merges all paths of a file into one poly-Bézier curve: two separate objects in one file give a long jump between them and a large error.

`tools/ch4f_figures.py`; the SVG files of `Example_Meshmerize_Me/Flow_Around_Starfish` (the outline was extracted by ContourizeMe from a NOAA photograph, Wikimedia Commons).

# SVG PATH DATA

| command | parameters | curve |
|---|---|---|
| `M`, `m` moveto | `(x y)+` | starts a sub-path |
| `L` lineto | `(x y)+` | linear |
| `H`, `V` | `x+`, `y+` | linear |
| `C` curveto | `(x1 y1 x2 y2 x y)+` | cubic |
| `S` smooth curveto | `(x2 y2 x y)+` | cubic |
| `Q` quadratic | `(x1 y1 x y)+` | quadratic |
| `T` smooth quadratic | `(x y)+` | quadratic |
| `A` elliptical arc | `(rx ry rot large sweep x y)+` | an arc |
| `Z` closepath | – | back to the start |

Lower case: coordinates relative to the current point. Extra coordinates repeat the command (`L40 50 60 70` = `L40 50 L60 70`; after `M` they are line-tos).

IB2d's nine SVG files hold the compact numbers, relative commands, the flipped $y$, and `S`/`s` directly after `M` (first control point = the current point); none has an arc, a `Q`, a `T` or a `transform`.

Traps a parser has to handle (not all are in IB2d's files: left):

- compact numbers: the optimised starfish writes `M974 117.411414s-.7739.588586-1.800593`, that is `s -0.7739 0.588586 -1 0.800593`;
- smooth curves (`S`, `T`): the first control point is the reflection of the previous curve's last control point about the current point (the current point itself after any other command), not a coordinate in the file;
- arcs: two one-digit flags that may be written without separators (`a5 5 0 01 10 0`);
- the y axis: SVG's origin is the top left corner, y downwards; the simulation's is the bottom left, y upwards.

Senter's design notes, section 7 and table 1; the SVG specification.

# BÉZIER CURVES AND THE DE CASTELJAU ALGORITHM

A Bézier curve of degree $n$ (SVG uses $n = 1, 2, 3$) is defined by control points $P_0, \ldots, P_n$:

$$\gamma(t) = \sum_{i=0}^{n} b_{i,n}(t)\, P_i, \qquad b_{i,n}(t) = \binom{n}{i}(1-t)^{n-i}\, t^i, \qquad 0 \le t \le 1;$$

it passes through $P_0$ and $P_n$ (in general through no other control point); the other points give direction and curvature. Its derivatives are Bézier curves of the differences:

$$\gamma'(t) = n \sum_{i=0}^{n-1} b_{i,n-1}(t)\,(P_{i+1} - P_i), \qquad \gamma''(t) = n(n-1) \sum_{i=0}^{n-2} b_{i,n-2}(t)\,(P_{i+2} - 2P_{i+1} + P_i).$$

de Casteljau: a stable way to evaluate $\gamma(t_0)$: $p_i^{(0)} = P_i$, $p_i^{(j)} = (1-t_0)\, p_i^{(j-1)} + t_0\, p_{i+1}^{(j-1)}$, and $\gamma(t_0) = p_0^{(n)}$, filling a triangle of repeated linear interpolations.

> Two corrections to the design notes: the second derivative reads $P_{i+2} - 2P_{i+1}P_i$ there (a $+$ is missing); and in its worked example `M20 150 L200 120 Q280 150 200 180 Z`, the quadratic ends at $(200, 180)$, not $(200, 80)$ as the text says twice (the notes' own test, section 15, prints 182.483 for both lines, the length of the closing line from $(200, 180)$; from $(200, 80)$ it would be 193.1).

Senter's design notes, sections 3–4 and 7.

# ARC LENGTH, AND THE THREE-POINT RULE

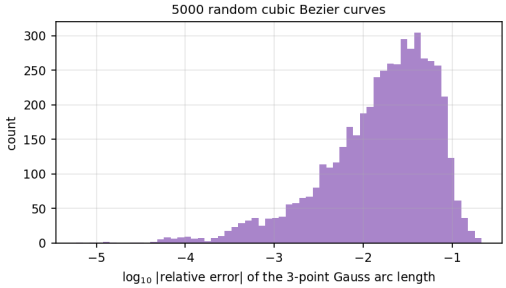
The number of points a curve needs is its length over $\Delta s$:

$$L(\gamma) = \int_0^1 \|\gamma'(t)\|\,dt = \int_0^1 \sqrt{B_x'(t)^2 + B_y'(t)^2}\;dt.$$

For a line, $L$ is the distance between the ends. The notes use three-point Gauss quadrature for the curves:

$$\int_0^1 f\,dt \approx \tfrac{1}{18}\Big[5f\big(\tfrac{1}{2} - \tfrac{1}{2}\sqrt{\tfrac{3}{5}}\big) + 8f\big(\tfrac{1}{2}\big) + 5f\big(\tfrac{1}{2} + \tfrac{1}{2}\sqrt{\tfrac{3}{5}}\big)\Big],$$

exact for polynomials of degree 5 or less. But $\|\gamma'\|$ is a *square root* of a polynomial, not a polynomial: on 5000 random cubic curves in a $1000 \times 1000$ box the rule's relative error has median 2.0 %, 95th percentile 8.9 %, maximum 21 %, and is above 1 % for 69 % of the curves. On ContourizeMe's tiny, nearly straight pieces (the starfish's 3996 cubics) it is fine: median $1.4 \cdot 10^{-6}$, maximum 1.2 %.




Use more nodes, or subdivide a curve (de Casteljau at $t = \frac{1}{2}$ gives the two halves) until the rule converges, or measure the length on a fine polyline, as `svg_to_vertex.py` does. The notes' own test (section 15) shows the error: 103.164 for the wing's quadratic, whose length is 104.648 ($-1.4$ %).

Senter's design notes, section 6; `tools/ch4f_figures.py`.

# FROM SVG SPACE TO THE SIMULATION BOX

SVG space $S = [0, w] \times [0, h]$ has its origin top left, $y$ down; the experiment $E = [0, L_x] \times [0, L_y]$ has it bottom left, $y$ up. The notes' affine map (written as a $3 \times 3$ matrix, as the SVG standard writes transforms; the notes print its translation entry as $h$, which is right only when $h = L_y$):

$$T : (x, y) \mapsto \Big( \frac{L_x}{w}\, x,\;\; L_y - \frac{L_y}{h}\, y \Big).$$

Example: $(10, 60)$ in $[0, 100]^2$ goes to $(0.1, 0.4)$ in $[0, 1]^2$. Bézier curves are invariant under affine maps, so control points can be mapped first and the curve meshed in $E$ (or meshed in $S$ with $\Delta s$ mapped back).

- MeshmerizeMe reads $L_x$, $L_y$ and $N_x$ from an `input2d` in the SVG's folder (IB2d's format; other keys are ignored) and sets $\Delta s = \frac{1}{2}\,\Delta x = L_x/(2N_x)$; it expects $\Delta x = \Delta y$ but not a square box. The `.vertex` file is named from `string_name`, whatever the SVG is called.
- The map $T$ stretches each axis separately: when $w/h \neq L_x/L_y$, the shape is distorted. Make the drawing's aspect ratio match the box, or map with one scale (`svg_to_vertex.py --fit uniform`, its default) and place the object afterwards.
- IB2d's SVGs do not start at the origin (the starfish's `viewBox` starts at $(102.08, 20.94)$): map $[x_{\min}, x_{\min} + w] \times [y_{\min}, y_{\min} + h]$, not $[x_{\min}, w] \times [y_{\min}, h]$ as the notes' section 12.1 has it (slide 17).

Senter's design notes, sections 11 and 12.1; Senter et al. (2021), section 2.2 and appendix B; the IB2d SVG files.

# MESHMERIZEME: POINTS AT EQUAL DISTANCES

For each Bézier curve $\gamma$, find parameters $0 \le t_1 < \cdots < t_n \le 1$ with equal Euclidean distances

$$d(t_i) = \|\gamma(t_{i+1}) - \gamma(t_i)\| = \Delta s, \qquad i = 1, \ldots, n-1,$$

by gradient descent on the mean squared relative error

$$J(t) = \frac{1}{2(n-1)} \sum_{i=1}^{n-1} \left( \frac{d(t_i) - \Delta s}{\Delta s} \right)^2, \qquad t^{\text{new}} = t^{\text{old}} - \alpha \nabla J(t),$$

with a variable learning rate $\alpha$ (the paper gives $\partial J / \partial t_j$ in full). The number of points is estimated as the arc length over $\Delta s$. All paths of the file are merged into one poly-Bézier path, split into sub-paths of equal arc length processed in parallel, each seeded with $n = L_{sp}/\Delta s$ points; the command line sets the learning rate, the convergence threshold, and the number of points per sub-path (`num-points`).
Errors: on 5000 SVG files with one random cubic each (a $1000 \times 1000$ pixel drawing onto a $1 \times 1$ box at $256 \times 256$, default settings), the median relative spacing error per curve averaged 3.2 %, its middle half 2.6–3.5 %. Strongly curved sub-paths may get too many points (use `num-points`); separate objects in one file get a large error at the jump between them.

Senter et al. (2021), section 2.2 and 2.2.1, equations (1)–(4) and figure 3.

# INSTALLING AND RUNNING IT

From the project's main folder (a clone of https://github.com/dmsenter89/MeshmerizeMe):

```
pip install .
# or, in a virtual environment with the
# supplied requirements:
pip install . --no-deps
pip install -r requirements.txt
```

Both scripts are put on the path (on Linux and Mac). Python 3; the paper names OpenCV, Tkinter (the GUI; part of Python, not a pip package) and Matplotlib; `requirements.txt` was not read here. "This version will only handle 2D code."

With the SVG and an `input2d` in one folder:

```
MeshmerizeMe.py simple.svg
# batch: a list of files, or a pipe
find . -name '*svg' | MeshmerizeMe
```

"That's it! Just make sure an input2d file is also present in the same folder as the SVG image." A minimal `input2d` for MeshmerizeMe needs only `Nx`, `Lx`, `Ly` and `string_name` (the paper's four lines, for $[0, 0.5]^2$ at $64 \times 64$); a simulation needs the rest of IB2d's keys. MeshmerizeMe's plotting tool shows the `.vertex` file to check the geometry. The wiki's script is `MeshmerizeMe.py`, the paper's `MeshmerizeMe`; the pipe line is from the notes' wish list (section 2), the batch mode and the pipe are confirmed by the paper.

The 2021 lecture, slides 4, 5 and 7 (with the project's documentation pages "Installation-Instructions", "The-ContourizeMe-Script", "The-MeshmerizeMe-Script"); Senter et al. (2021), section 2; the batch line from Senter's notes, section 2.

# WHAT IB2D SHIPS: EXAMPLE_MESHMERIZE_ME

| folder | contents |
|---|---|
| `Dragonfly_Wing` | the paper's first example: `Images/` (the cropped wing as png and svg, the Chan–Vese contour), `Wingitize_Me.m`, `wing_flow.vertex` (MeshmerizeMe's points), `wing_vein.vertex`/`.target` (the simulation's), `please_Compute_External_Forcing.m`, `input2d`, `main2d.m` |
| `Insect_Wing_Veins` | the same case, copied earlier (2018): the same `Wingitize_Me.m`, `.vertex` and `.target` files; `cropped_wing.svg` and its optimised version; $\mu = 0.1$, $T = 1$ (Dragonfly: 0.01, 10) |
| `Flow_Around_Starfish` | the first starfish (May 2019, "still working out kinks"), ported here: `starfish_initial.svg`, `_cleaned.svg`, `_final.svg`, `star_flow.vertex` (MeshmerizeMe's points), the MATLAB script that places them in a channel (`modify_read_Vertex_Points_and_Plot_Them_and_Print_Them.m`), `starfish_flowtank.vertex`/`.target`, the forcing, a VisIt session, and `MeshingTutorial.md` |
| `Starfish` | the paper's third example (its section 3.3; August 2019): the same SVGs and `star_flow.vertex`; the starfish at $x = 0.5$ on targets $(5 \cdot 10^5)$ and springs, a 2 Hz sine ($U_{\max} = 500$, $w = 0.23$), $T = 3$ |
| `old`, top level | a leaf: `optimal_leaf.svg`, `Leafitize_Me.m`, `leaf_flow.vertex`, `leaf_vein.vertex`/`.target` |

All MATLAB (Octave cannot run matIB2d, chapter 4a). `MeshingTutorial.md` is unfinished: it finds the image, runs ContourizeMe, "play with the levers until the contour seems to match", opens the SVG in Illustrator or Inkscape, and stops mid-sentence at the clean-up (a last heading holds only a plot of `star_flow.vertex`).

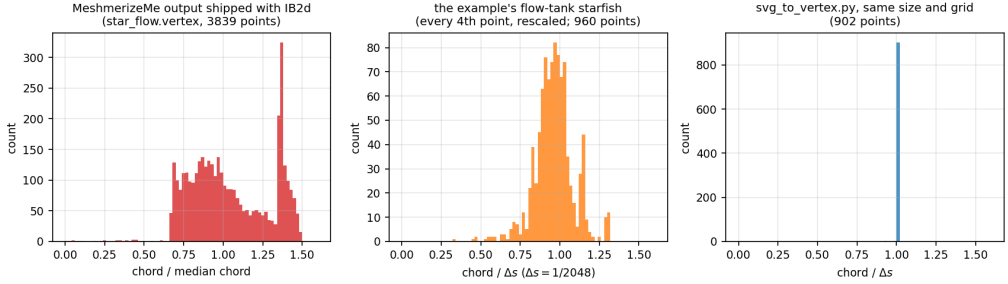
# THE STARFISH EXAMPLE'S MATLAB PIPELINE, READ

`modify_read_Vertex_Points_and_Plot_Them_and_Print_Them.m` of `Flow_Around_Starfish` turns MeshmerizeMe's `star_flow.vertex` (3839 points) into the channel case (the `Starfish` folder's copy differs in step 3 and the target stiffness, and adds springs: slide 14):

1. scale the points so that their *mean* spacing becomes $\Delta s = L_x/(2N_x) = 1/2048$ (for $1024 \times 1024$ on $[0,1]^2$), then by $0.25$ (a smaller starfish);
2. keep every `skip`-th point, `skip = ceil(ds_Desired/ds_Check)` $= 4$: 960 points, spacing back near $\Delta s$ on average (the ratio is 4 only up to rounding: computed with NumPy's `mean` it is $4 + 9 \cdot 10^{-16}$ and `ceil` gives 5, 768 points; MATLAB's rounding went the other way, as the 960 shipped points show);
3. move the centroid (the mean of the points) to $(0.3125, 0.125)$: a starfish $0.080 \times 0.074$ at mid-height of the $1 \times 0.25$ channel, a third of the way along;
4. add two walls at $y = 0.01$ and $0.24$ with points $2\,\Delta s$ apart (1025 each, from $x = 0$ to 1, two places that are one in the periodic box: each wall has a doubled point), and write targets: $2.5 \cdot 10^5$ on the starfish, $2 \cdot 10^7$ on the walls (MATLAB's 1-based indices).

> Three things to know: the starfish's spacing is whatever MeshmerizeMe's was, thinned (next slide); the walls' points are $2\Delta s = \Delta x$ apart, one per cell of the $1024 \times 256$ grid, not the two per cell of the rule; and the hard-coded $L_x = L_y = 1$, $N_x = 1024$ of step 1 must match the flow tank's $\Delta x$ ($1/1024$ there too) by hand.

# HOW EVEN ARE THE POINTS?




MeshmerizeMe's `star_flow.vertex` (3839 points, one per path piece of the cleaned SVG: its nodes, slide 17): chords from 0.015 to 1.51 times their median, 5th–95th percentile 0.70–1.42. The example's flow-tank starfish (every fourth point, rescaled): 0.34–1.31 $\Delta s$, 5th–95th 0.77–1.15.

`svg_to_vertex.py` on the same SVG, the same size and grid: 902 points, every chord 1.001 $\Delta s$ but the closing one (0.92). The shipped file's spread does not match the paper's 3.2 % typical error; it may predate the version the paper describes (slide 17).

`tools/ch4f_figures.py`.

# WHAT THE SHIPPED `star_flow.vertex` IS

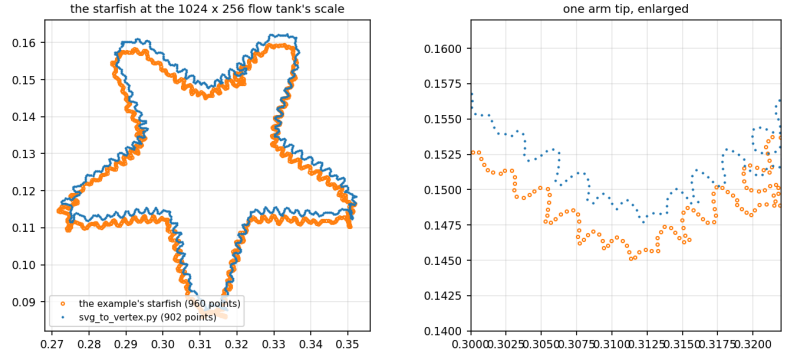
- Its 3839 points are exactly the first points of the 3839 pieces of `starfish_final.svg`, mapped by

$$x' = 0.5\,\frac{x - 102.078}{1217.029 - 102.078}, \qquad y' = 0.5\,\frac{1152.080 - y}{1152.080 - 20.939}.$$

- The four numbers are the drawing's `viewBox`: $x_{\min}$, $y_{\min}$, width, height. The map reads them as $x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$, the reading of the notes' section 12.1 ("$[x_{\min}, \text{width}] \times [y_{\min}, \text{height}]$"), into a $0.5 \times 0.5$ box (the box of the plot in `starfish_mesh_image.png`). It stretches $x$ by 1.45 % against $y$, and the MATLAB example inherits the stretch.
- At $256 \times 256$ on $[0, 0.5]^2$ (the MATLAB script's note: "original resolution set at 256x256") $\Delta s = 0.5/512$, and no piece is longer than 1.02 $\Delta s$ (median 0.67 $\Delta s$): a count of points made path by path, arc length over $\Delta s$ as the paper estimates it (section 2.2), is at most about one per piece, and the file has exactly one per piece. The version the paper describes merges the paths first (section 2.2.1). The file was committed in May 2019 and the paper received in November 2019: it may predate that version.
- The lesson: plot and measure the points you are given (chord/$\Delta s$, previous slide), not only the outline they follow.

`star_flow.vertex` and `starfish_final.svg` of `Flow_Around_Starfish`; Senter's notes, section 12.1; Senter et al. (2021), section 2.2.

# THE TWO POINT SETS ON TOP OF EACH OTHER




Same outline and size (orange: the example's 960 points; blue: `svg_to_vertex.py`'s 902), but the MATLAB script centres the mean of its points and the sampler its bounding box, so the blue set sits 0.0025 higher, and the orange one is 2 % wider for its height (slide 17). Enlarged at the notch between the two upper arms (right), the orange points bunch and gap; the blue ones are $\Delta s$ apart along the traced outline. Both follow the SVG; neither is more "accurate" than the tracing, which has pixel-scale wiggles of its own.

`tools/ch4f_figures.py`.

# A SAMPLER OF OUR OWN: SVG_TO_VERTEX.PY

1. Read every `<path>` (and lines, polylines, polygons, rectangles without rounded corners, circles, ellipses); parse M L H V C S Q T A Z, absolute and relative, with the traps of slide 8; arcs become cubics.
2. Join consecutive pieces whose start is the previous end: ContourizeMe's thousands of pieces become one curve per outline.
3. Map to $[0, L_x] \times [0, L_y]$, one scale by default (`--fit stretch` for the notes' map), then optionally `--size` and `--center`.
4. Walk along each finely sampled curve placing the next point at Euclidean distance $h$ from the last (a circle–segment intersection); find $h \approx \Delta s$ by bisection so the last chord, to the end or back to the start, is $h$ too; retry other starting points, and fall back to equal arc length if a hook prevents an even close (it says so).

```
python3 svg_to_vertex.py drawing.svg --plot
python3 svg_to_vertex.py \
    starfish_final.svg --input2d input2d \
    --size 0.0803 --center 0.3125,0.125
```

| SVG (unit box, $256^2$) | points | chord/$\Delta s$ |
|---|---|---|
| the notes' wing | 801 | 1.0005 |
| circle, $r = 0.4$ | 1287 | 0.9999 |
| starfish (final) | 2469 | 0.9996 |
| dragonfly wing (Images) | 4253 | 0.965–1.010 |
| insect wing veins | 5038 | 0.992–1.038 |
| leaf (old) | 2321 | 0.32–1.001 (1 fallback) |

The circle's 1287 points match $2\pi r/\Delta s = 1286.8$. Step 4's last chord is $h$ within the script's 0.8–1.25 $h$ (the starfish: 0.92 and 1.13 $\Delta s$, slides 16, 20); step 2 joins ends within $10^{-5}$ of the drawing's size (`--merge-tol`). The (optimised) wing veins need it: rounding left their pieces' ends up to 0.001 apart; at $10^{-6}$, the first default, they stayed 240 open pieces (5245 points, chords 0.85–1.16 $\Delta s$); joined, 34 contours, as in the unoptimised file.

`examples/IB2d/Mesh_From_SVG/svg_to_vertex.py`; NumPy only (Matplotlib for `--plot`).

# THE STARFISH IN A CHANNEL, IN PYTHON

`examples/IB2d/Mesh_From_SVG/Starfish_Channel`, a port of the MATLAB example `Flow_Around_Starfish` at half its resolution:

- `make_starfish_channel.py path/to/starfish_final.svg`: the outline sampled at $\Delta s = 1/1024$ for $512 \times 128$ on $[0,1] \times [0,0.25]$, sized to $0.0803$ and centred at $(0.3125, 0.125)$ as in MATLAB, but by its bounding box (MATLAB: the mean of its points, 0.0025 lower): 405 points; walls at $y = 0.01$ and $0.24$, $\Delta s$ apart (2 $\times$ 1024 points); targets $2.5 \cdot 10^5$ and $2 \cdot 10^7$;
- `please_Compute_External_Forcing.py`: the MATLAB forcing with the same numbers: in $0.04 \le x \le 0.06$, $0.02 \le y \le 0.22$, a penalty $k = 10^4$ towards $u_x = -U_{\max} \sin(2\pi \cdot 5\,t)\,(\frac{L_y}{2} + \frac{w}{2} - y)(\frac{L_y}{2} - \frac{w}{2} - y)$, $U_{\max} = 2500$, $w = 0.25$, during a 0.2 s pulse, then 0.2 s at rest: $39 \sin(10\pi t)$ on the centre line;

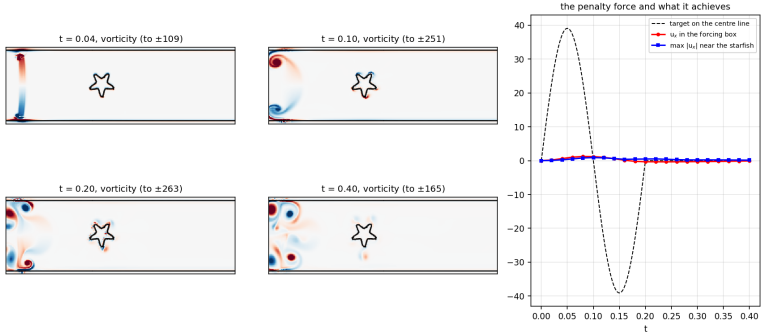
`input2d`: $\rho = 1000$, $\mu = 0.1$, $\Delta t = 5 \cdot 10^{-5}$, $T = 0.4$ (one pulse and one rest; MATLAB's runs to 1).

8000 steps, 223 s (under four minutes) on the laptop.

The SVG is not copied into this repository: it is part of IB2d (GPL-3.0) and was extracted by ContourizeMe from a NOAA photograph; the script reads it from your IB2d clone. `main2d.py` (IB2d's own file) and the forcing (a port of IB2d's) stay under IB2d's GPL-3.0.

`main2d.py` is the Turek–Hron example's (any pyIB2d run script with external forcing works); the files carry 0-based indices.

# THE RUN: VORTICITY, AND WHAT THE PENALTY ACHIEVES




Left: vorticity at $t = 0.04, 0.10, 0.20$ and $0.40$ (each frame on its own scale): the forcing box drives a jet that rolls up into vortex pairs at the walls' ends of the box, a wake forms behind the starfish during the pulse, and the vortices drift back during the rest. Right: the centre-line target (dashed), the speed reached in the forcing box (red) and the largest $|u_x|$ near the starfish (blue).

The target peaks at 39 at $t = 0.05$; the box reaches 1.23 at $t = 0.08$, 3 % of the target and some 0.04 s late; during the rest the box's centre line flows backwards at about $-0.35$ while the channel is still. Near the starfish $|u_x|$ peaks at 0.89 at $t = 0.10$: with the paper's $L = 0.08$, $Re = \rho LU/\mu \approx 710$ for that local speed, 250 for the flow arriving (slide 22); the paper's 800 (whose $V = 1$ m/s is "half the maximum oscillatory inflow speed") is for its 2 Hz run. 8000 steps, 223 s.

# WHY THE FLUID REACHES 1, NOT 39

- The time scale. The forcing adds $-k(u - u_T)$ to $\rho\, Du/Dt = -\nabla p + \mu\Delta u + f$: on its own it relaxes a parcel of fluid in $\rho/k = 0.1$ s, half the pulse. A parcel pulled so towards $39 \sin 10\pi t$ would reach 16.5 at $t = 0.086$, not 1.23.
- The channel. The fluid in the box cannot speed up alone: between the walls the flux is the same at every $x$ (0.0685–0.0715 at $t = 0.10$), so the box ($10 \times 102$ cells, $0.02 \times 0.2$) has to drive the whole channel ($1 \times 0.23$). Its momentum, $\rho A_{\text{channel}}\, d\bar{u}/dt \approx k A_{\text{box}}(\bar{u}_T - \bar{u}_{\text{box}})$, has the time scale $\rho A_{\text{channel}}/(kA_{\text{box}}) \approx 6$ s and gives a mean speed of 0.32 at $t = 0.1$; the run has 0.31. The pressure spreads the box's force over all that fluid.
- The box. Its centre line reaches 1.23 with the fluid beside it, next to the walls, flowing back ($-1.6$ to $-2.3$ at $t = 0.08$, $x = 0.05$); downstream the flow is a plug at about 0.3. During the rest the box's centre line stays near $-0.35$ while the channel is still (mean speed $-0.01$): a recirculation.
- $U_{\max}$. The paper's (A.8) divides the parabola by $w^2/4$, which makes $U_{\max}$ the centre-line target; the code does not, so 2500 means 39. The paper's own folder, `Starfish`, uses $U_{\max} = 500$, $w = 0.23$: 6.6 on the centre line, at 2 Hz. How either value was chosen is not recorded.
- The Reynolds number, with the paper's $L = 0.08$: 710 with the largest $|u_x|$ near the starfish (0.89), 250 with the flow arriving at it (0.31), 490 with the paper's own $V$, half the largest inflow speed (half of 1.23). The paper's 800 ($V = 1$ m/s) is for its 2 Hz run in `Starfish`.

The run's `uX` dumps (slide 21); Senter et al. (2021), (A.6)–(A.8); the forcing files of `Flow_Around_Starfish` and `Starfish`.

# FOUR THINGS THE STARFISH EXAMPLE TEACHES

1. A penalty is not a boundary condition. $-k(u - u_T)$ with $k = 10^4$ acting on a fluid of density $10^3$ relaxes a parcel of fluid on a time scale $\rho/k = 0.1$ s, half the pulse, which alone would allow about 16 of the 39; but the small box has to drive the whole channel (slide 22): the target is 39, the fluid reaches about one. How the shipped $U_{\max} = 2500$ was chosen is not recorded (the code, unlike the paper's (A.8), does not divide by $w^2/4$). Change $\rho$, $k$ or the box and the realised speed changes; measure it (slide 21).
2. The paper and the files differ: the paper describes oscillatory flow at 2 Hz, and the folder it names, `Starfish`, has it; the example ported here, `Flow_Around_Starfish`, committed three months earlier "still working out kinks", pulses at 5 Hz with rests, as above. Read the example's files, not only its description.
3. Dump 0 of the velocity is transposed in Python IB2d: `u.0000.vtk`, `uX.0000.vtk` and `uY.0000.vtk` have `DIMENSIONS` $N_y\ N_x$, later dumps $N_x\ N_y$ (`IBM_Driver.py` line 668 passes `U`, `V` untransposed where line 848 passes `U.T`, `V.T`; `Omega`, `uMag` and `P` are fine). That holds with the `vtk` package, as here; pyIB2d's plain-text writer gets `u.*.vtk` the other way round. The values are zero at $t = 0$, but on a non-square grid VisIt shows dump 0 with the wrong extent and a reader that trusts the header gets the wrong shape.
4. Plot and measure what you are given. The shipped `star_flow.vertex` is the drawing's nodes, one per path piece, mapped with the `viewBox` misread (slide 17): its spacing came from the drawing, not from $\Delta s$.

# A PROTOCOL FOR A GEOMETRY OF YOUR OWN

1. Image: high contrast, little noise, the object whole; crop it to what you will simulate (the dragonfly example opened some veins so flow has an entry and an exit).
2. Contours: ContourizeMe (or trace by hand in Inkscape); check the number of contours and their lengths, and delete the noise.
3. One object, one path; simplify and flatten transforms before saving.
4. Points: MeshmerizeMe or `svg_to_vertex.py` with the simulation's $L_x$, $N_x$; check the chord statistics (min, median, max of chord$/\Delta s$) and plot the points over the SVG.
5. Scale and place with one scale (no stretching), in physical units; size the box so the object is far enough from the periodic edges.
6. Physics: targets for rigid parts, springs and beams for elastic ones (chapter 4c's generators), walls at $\Delta s$ too, and the flow (forcing box, its realised speed measured).
7. Resolution: re-run the points for a finer grid rather than thinning or rescaling a fine set (chapter 4d).

# EXERCISES

1. Trace a leaf, a fin or a wing of your own in Inkscape over a photograph; run `svg_to_vertex.py` at $64^2$, $128^2$ and $256^2$ and plot the points over the outline. How many points does the outline get at each grid?
2. Run `svg_to_vertex.py --fit stretch` on the starfish in a $1 \times 0.25$ box and compare its aspect ratio with `--fit uniform`'s.
3. Write the three-point Gauss rule and a subdividing version (split at $t = \frac{1}{2}$ until two levels agree to $10^{-6}$); compare their errors on 1000 random cubics.
4. Re-run the starfish channel with $k = 10^5$ and $10^6$ in the forcing (and $\Delta t$ halved if needed); plot the realised centre-line speed against the target. What $U_{\max}$ gives a peak of 1?
5. Put three starfish in the channel (the paper's figure 10) by copying and translating the point set; compare the wake of the middle one with a single starfish.
6. Give the starfish springs and beams between neighbours instead of targets (soft arms), keeping one target at its centre; what does the flow do to the arms? (IB2d's `Starfish` folder already gives its starfish springs, $10^7$ between neighbours and $10^5$ across, on top of the targets.)

# SUMMARY

- MeshmerizeMe turns images into IB geometry in two steps: ContourizeMe finds contours (filters, thresholds, OpenCV contours, Chan–Vese, Bézier curves to SVG); MeshmerizeMe reads the SVG and `input2d` and writes points about $\Delta x/2$ apart. A vector editor in between cleans the SVG, or replaces ContourizeMe altogether.
- SVG paths are poly-Bézier curves; parse them carefully (compact numbers, smooth curves, arcs, the flipped $y$), measure lengths on fine polylines rather than with a three-point rule, and map with one scale.
- MeshmerizeMe solves for equal Euclidean distances by gradient descent (3 % typical error in the paper); the starfish output shipped with IB2d is far less even (its points are the SVG's nodes, slide 17), and the example thins it further. A walk with bisection gives equal chords $h$ within $1/(2n)$ of $\Delta s$ (1.001 $\Delta s$ for the 902-point starfish), the closing one within the script's 0.8–1.25 $h$.
- From points to a simulation: targets, walls at $\Delta s$, a forcing whose realised speed you measure. The Python port of the starfish ran in 223 s (under four minutes); the penalty reached about 3 % of its target speed in the box, the channel's mean speed less than 1 % (slide 22).
- Part II ends here. In reading order Part III (IBAMR, chapter 5) comes next; the VisIt part continues with chapter 27.

# REFERENCES AND CREDITS


- D. M. Senter, D. R. Douglas, W. C. Strickland, S. G. Thomas, A. M. Talkington, L. A. Miller, N. A. Battista, “A semi-automated finite difference mesh creation method for use with immersed boundary software IB2d and IBAMR”, Bioinspir. Biomim. 16 (2021) 016008, https://doi.org/10.1088/1748-3190/ababb0. Its figures are not reproduced here; figure 1’s workflow is redrawn.
- M. Senter, “Creating a Curvilinear Mesh from Bézier Curves”, design notes for an early C++ version of MeshmerizeMe (UNC Chapel Hill, about 2017).
- MeshmerizeMe: https://github.com/dmsenter89/MeshmerizeMe and its documentation pages (installation, ContourizeMe, MeshmerizeMe), as presented in the 2021 slides “MeshMerizeMe”.
- IB2d: https://github.com/nickabattista/IB2d, `matIB2d/Examples/Example_Meshmerize_Me` (clone at commit 58663e1 of 2024-03-06), and `pyIB2d` for the run. The starfish outline was extracted by ContourizeMe from a photograph by J. P. McVey (`Reef0297.jpg`) of the NOAA Sea Grant Program’s Coral Kingdom Collection (Wikimedia Commons), as the paper credits (its reference 47; `MeshingTutorial.md` links the same file).
- This chapter’s code (October 2026): `examples/IB2d/Mesh_From_SVG` (`svg_to_vertex.py`, `Starfish_Channel/`, README), `tools/ch4f_figures.py`, `wsl/final/ch4f_star.sh`.
- The SVG 1.1 specification (paths, the elliptical-arc implementation notes): https://www.w3.org/TR/SVG11/.